Lab 1: Introduction to Natural Language Processing

Session 4: Contextual word-embeddings for text representation.

Goal

The main goal of this session is to explore the techniques utilized to generate dense vector representations of words, commonly known as word embeddings. It aims to familiarize students with deep learning-based techniques that generate contextual word-embedding representations. Particularly, this session explores transformer-based models (BERT, RoBERTa). Moreover, the session reviews some models pre-trained in the Spanish language.

Students can build document representations based on these techniques, thus enhancing their understanding and proficiency in text representation for addressing natural language processing tasks.

Bibliography

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.
2. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., 2017. Attention is all you need. Advances in neural information processing systems, 30.
3. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. NAACL-HLT 2019, pages 4171–4186, ACL.
4. Liu, Yinhan, et al. "Roberta: A robustly optimized bert pretraining approach." *arXiv preprint arXiv:1907.11692* (2019).
5. Cañete, J., Chaperon, G., Fuentes, R., Ho, J. H., Kang, H., & Pérez, J. (2020). Spanish pre-trained bert model and evaluation data. In Proceeding of the Practical ML for Developing Countries Workshop @ ICLR 2020, pages 1-9, Addis Ababa, Ethiopia.
6. Gutiérrez-Fandiño, A., Armengol-Estapé, J., Pàmies, M., Llop-Palao, J., Silveira-Ocampo, J., Pio Carrino, C., Armentano-Oller, C., Rodriguez-Penagos, C., Gonzalez-Agirre, A., & Villegas, M. (2022). MarIA: Spanish Language Models. Procesamiento del Lenguaje Natural, 68, 39-60.
7. https://huggingface.co/docs/transformers/index

WORD EMBEDDINGS

Word embeddings are numerical representations of words in a continuous vector space, where words with similar meanings are positioned closer to each other. Related words like "*women*" and "*men*" are assigned to very close n-dimensional vectors, while different words like "*men*" and "*overweight*" have distant vectors. The meaning of a word can be reflected in its vector (embedding),

and a model can use this information to learn the relationship between words. Word embedding methods are based on *the principle of distributional semantics*, which assumes that words occurring in similar contexts have similar meanings. These embeddings capture semantic relationships between words, allowing machines to interpret and process language more effectively. It can be distinguished between two main types of word embeddings: contextual and static. Static embeddings were introduced in the previous lab session (Lab1-S3: Static word-embeddings por text representation).

Contextual Word Embeddings (CWE)

On the other hand, contextual embeddings consider each word's surrounding context to generate its embedding. Models such as BERT (Bidirectional Encoder Representations from Transformers) and all BERT-based families are prominent examples. By using deep learning architectures like transformers, **contextual embeddings capture the dynamic nature of language, providing richer representations that adapt to the context of each word within a sentence**.

ALGORITHMS FOR BUILDING CONTEXTUAL WORD EMBEDDING

BERT is described as one of the most significant breakthroughs in the history of (Google's) Search [2]. Specifically, it is used in the disambiguation of queries in all searches in English, but other languages are also included, such as Spanish, Portuguese, Hindi, Arabic, German, etc. [3]. BERT has also been used in many automatic text classification tasks and other Natural Language Processing tasks.

The general idea of building a BERT-based classifier consists of two essential stages, as seen in Figure 1. The first is to obtain the (deep) representation of the texts from the input data and use this representation as input to a classifier, which is usually a Multi-Layer Perceptron (MLP) that receives the output of the BERT model as input and outputs a layer with as many neurons as classes in the task.

It is essential to highlight that the BERT model can be used directly from the general data with which it was trained (pre-trained), or it

can be fine-tuned to consider specific aspects of the task to be solved (fine-tuned).

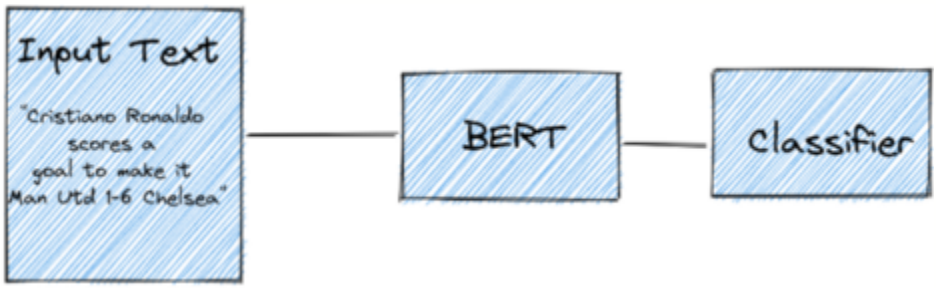

Fig 1. A general architecture for using BERT in classification tasks

The BERT architecture consists of stacked encoders (see Figure 2). Each encoder encapsulates two sublayers: a self-attention layer and a feed-forward layer (see Figure 3).

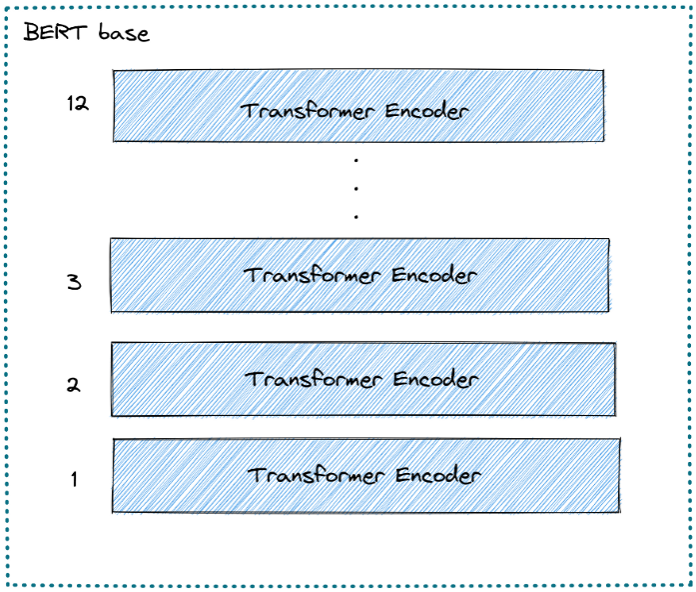

Fig 2. BERT architecture

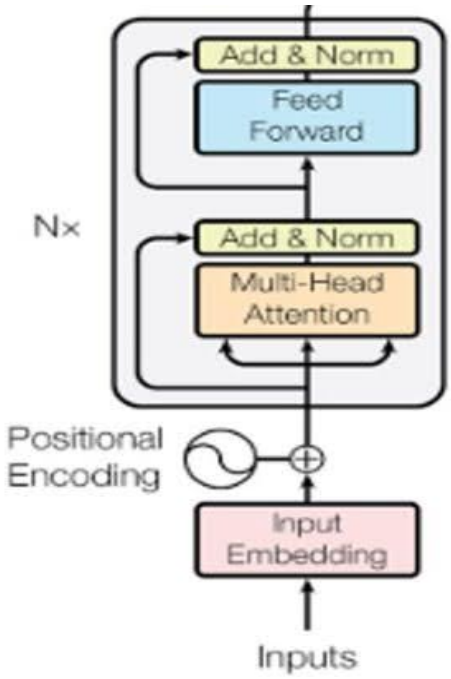

Fig. 3 Transformer Encoder

There are different models of BERT regarding the size of the architecture:

- Base BERT is a model consisting of 12 encoder layers, 12 self-attention heads, 768 hidden sizes, and 110 million parameters.
- Large BERT is a model consisting of 24 encoder layers, 16 self-attention heads, 1024 hidden sizes, and 340 million parameters.

There are two reasons why BERT is a robust language model:

1. It is pre-trained with unlabeled data from BooksCorpus (800 million words) and Wikipedia (2.5 billion words).
2. It is pre-trained using the bidirectional nature of stacked encoders. This fact means that BERT learns the information in a sequence of words from left to right and from right to left.

How does BERT process the input data?
Given the sentence below, the first step would be to get the tokens (words) of the sequence.

"I will watch memento tonight"

The BERT model requires a sequence of tokens (words) as its input. Within each token sequence, there are two special tokens that must be included according to BERT's expectations (see Figure 4):

- [CLS]: This token marks the beginning of each sequence and is known as the classification token.

- [SEP]: This token serves the purpose of informing BERT about the boundaries between sentences within the token sequence. It plays a crucial role, especially in tasks involving next sentence prediction or question answering. In cases where there is only one sentence, this token is added at the end of the sequence.

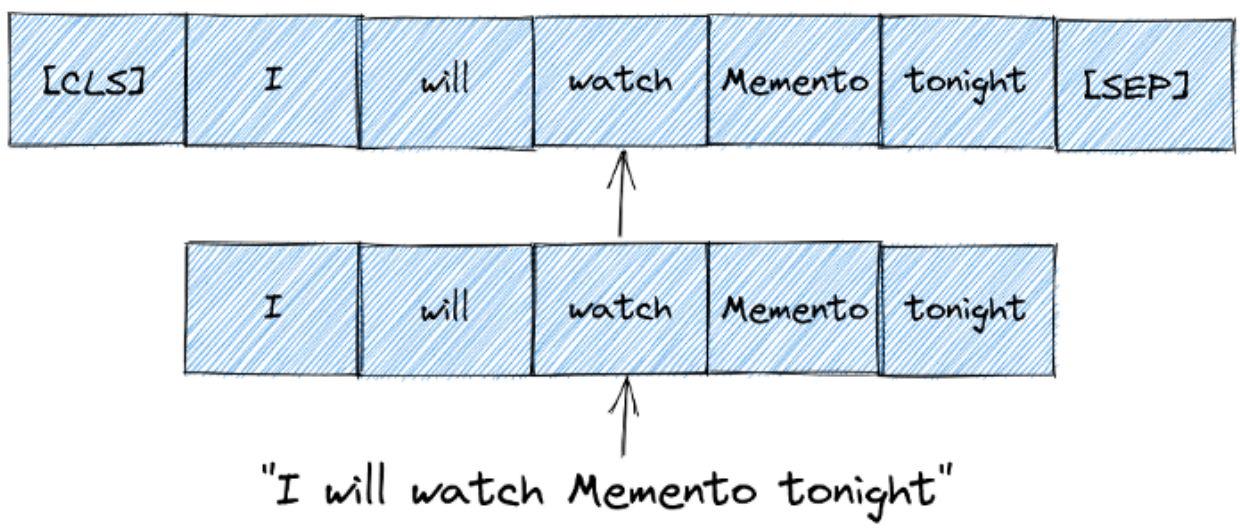

Fig 4. Tokenized textual input

Transforming the input sentence into a token sequence for BERT requires just a single line of code. To achieve this, we utilize the BertTokenizer class, which will be described later. It is crucial to consider that the maximum allowable token length for the BERT model is 512. If a sequence's tokens fall short of this limit, sequence completion with the [PAD] token can be applied. However, if the token sequence surpasses the 512-token limit, truncation is performed accordingly.

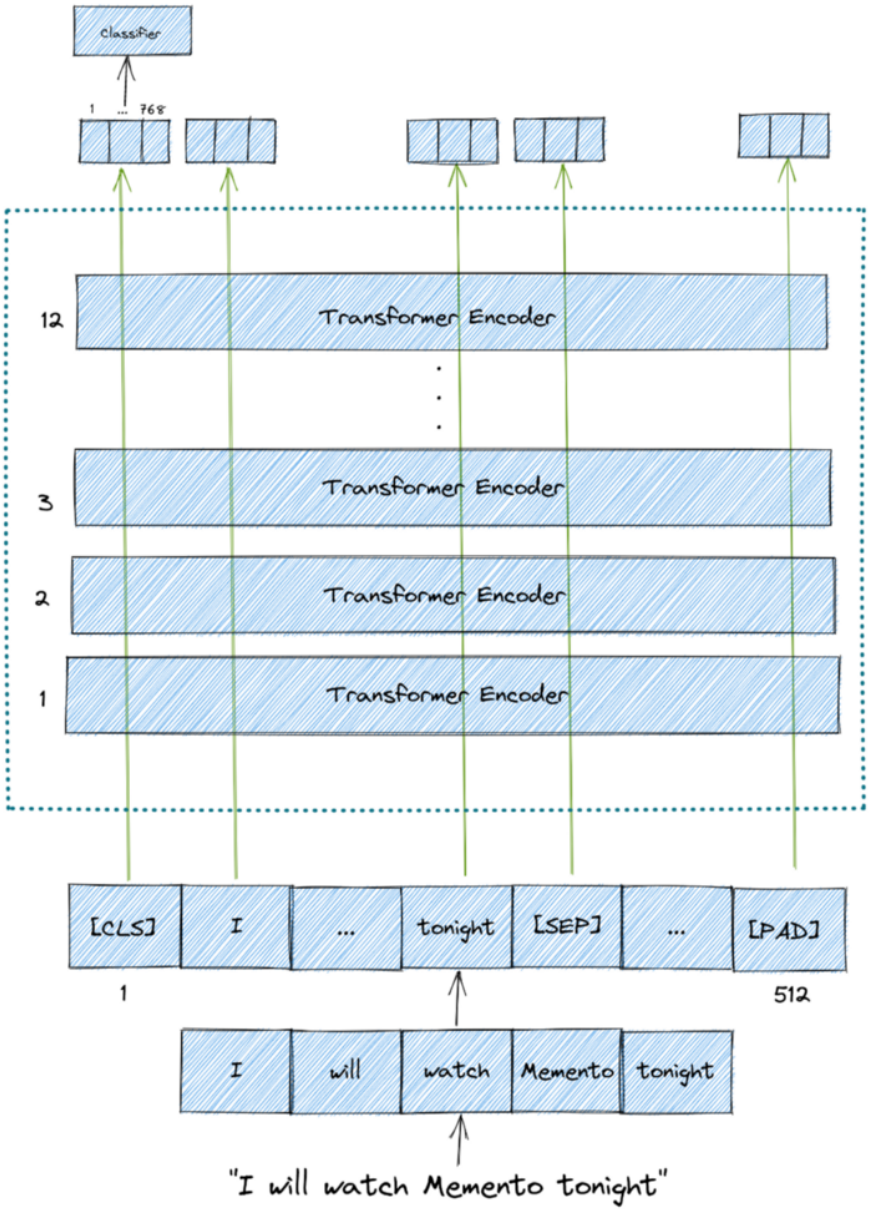

Fig 5. BERT output for textual input.

The BERT model then generates a 768-dimensional embedding vector for each token. These vectors can serve as input for various Natural Language Processing applications, including text classification, next sentence prediction, Named Entity Recognition (NER), and Question & Answering.

In the case of text classification, we focus on the embedding vector output of the special token [CLS]. This entails utilizing the 768-dimensional embedding vector of the [CLS] token as input for the classifier (MLP). The classifier, in turn, generates a vector of a size corresponding to the number of classes in the classification task. This process is illustrated in Figure 5.

EXAMPLE 1: Tokenization with BERT Tokenizer

```
import torch
from transformers import BertTokenizer
tokenizer=     BertTokenizer.from_pretrained('bert-base-uncased')
#The max_length parameter depends on the texts' length in the dataset
#Examples of the tokenization performed by BERT·
example_text = ["Yo it shirt he gave New York was funny.", "my husband is sick, homemade chicken soup loading.", "Witch raises wind to break up enemy’s lumber pound.", "i got a new shirt at work at it is the WORST material ever", "wave is so poor the girls can’t even dress up for Halloween", "The theory of paint indicates feelings of isolation in society.", "Wish sometimes I had access to the ever elusive cock carousel..", "off probation tomorrow ima be a free woman again, FUCK THE SYSTEM"]
tokenized_text = tokenizer.tokenize(example_text[0])
```

```
print(tokenized_text)
tokenized_text = tokenizer.tokenize(example_text[1])
print(tokenized_text)
bert_input = tokenizer(example_text,
padding='max_length', max_length = 30, truncation=True,
return_tensors="pt")
# Transform tokens to vocabulary indexes
indexed_tokens =
tokenizer.convert_tokens_to_ids(tokenized_text)
#BERT WORKS WITH THESE REPRESENTATIONS OBTAINED FOR THE
TOKENS
print(bert_input['input_ids'])
print(bert_input['token_type_ids'])
print(bert_input['attention_mask'])
#To transform the sequence of token indices to words in
the texts
example_text = tokenizer.decode(bert_input.input_ids[0])
print(example_text)
example_text = tokenizer.decode(bert_input.input_ids[1])
print(example_text)
```

You can change or choose the most suitable runtime environment (GPU or CPU) usint `torch`

```
import torch
if torch.backends.mps.is_available():  # Mac M? GPU
    device = torch.device("mps")
elif torch.cuda.is_available():  # Nvidia GPU
    device = torch.device("cuda")
else:  # CPU
    device = torch.device("cpu")
```

EXAMPLE 2: Obtaining contextualized word embeddings using the pre-trained BERT model

```
import torch
from transformers import BertTokenizer, BertModel
```

```
tokenizer=BertTokenizer.from_pretrained('bert-base-uncased')
example_text = ["Yo it shirt he gave New York was funny.", "my husband is sick, homemade chicken soup loading.", "Witch raises wind to break up enemy's lumber pound.", "i got a new shirt at work at it is the WORST material ever", "wave is so poor the girls can't even dress up for Halloween", "The theory of paint indicates feelings of isolation in society.", "Wish sometimes I had access to the ever elusive cock carousel..", "off probation tomorrow ima be a free woman again, FUCK THE SYSTEM"]

#The max_length parameter depends on the texts' length in the dataset
bert_input = tokenizer(example_text, padding='max_length', max_length = 100, truncation=True, return_tensors="pt")
model = BertModel.from_pretrained('bert-base-uncased')
#Setting the evaluation mode, this option does not make gradient updating
model.eval()

# Send the data to GPU
# bert_input = bert_input.to('cuda')
# model.to('cuda')

# Send the data to mps, cuda or cpu
bert_input = bert_input.to(device)
model.to(device)
with torch.no_grad():
  outputs = model(**bert_input)
  # Transformers models always return tuples.
  # Here, the first element corresponds to the vectors in the output of the last BERT layer.
  encoded_layers = outputs[0]
  print(encoded_layers.size())
```

```
  #Here we obtain the embedding of the CLS tokens for
each input text.
  #This representation serves as a contextual embedding
of the texts.
  cls_vector = encoded_layers[:,0,:]
  print(cls_vector.size())
  #Vector associated with the CLS token of the first text
in the entry.
  cls_vector = cls_vector.cpu().detach().numpy()[0]
  print(len(cls_vector))
```

RoBERTa (Robustly Optimized BERT Pre-training Approach) [4] is a variant of the popular BERT language model introduced by researchers from Facebook AI Research in 2019. RoBERTa was trained with a larger batch size of 8,000 samples, compared to 256 for BERT. This allowed for better Graphics Processing Unit (GPU) memory utilization and faster training. RoBERTa was trained for longer, with more computational resources, resulting in improved performance. Conversely to BERT, which was trained on both Masked Language Modeling (MLM) and Next Sentence Prediction (NSP) tasks. RoBERTa removed the NSP task and focused solely on the MLM task, which proved to be more effective. Moreover, the model used a larger vocabulary of 50,000 tokens, compared to 30,000 for BERT, allowing for better representation of rare words. The increase in vocabulary size is accompanied by the use of a more efficient tokenizer, BPE (Byte Pair Encoding) in contrast to BERT's WordPiece. Instead of using the same masking pattern for all examples, RoBERTa used a different masking pattern for each example during training, which improved generalization.

These modifications, along with other minor changes, resulted in RoBERTa achieving significantly better performance than BERT on a wide range of Natural Language Processing tasks, such as

question answering, text classification, and language inference, while using the same architecture as BERT.

EXAMPLE 3: Tokenization with RoBERTa Tokenizer

```
import torch
from transformers import RobertaTokenizer
tokenizer=     RobertaTokenizer.from_pretrained(roberta-base')
#The max_length parameter depends on the texts' length in the dataset
#Examples of the tokenization performed by RoBERTa
example_text = ["Yo it shirt he gave New York was funny.", "my husband is sick, homemade chicken soup loading.", "Witch raises wind to break up enemy’s lumber pound.", "i got a new shirt at work at it is the WORST material ever", "wave is so poor the girls can’t even dress up for Halloween", "The theory of paint indicates feelings of isolation in society.", "Wish sometimes I had access to the ever elusive cock carousel..", "off probation tomorrow ima be a free woman again, FUCK THE SYSTEM"]
tokenized_text = tokenizer.tokenize(example_text[0])
print(tokenized_text)
tokenized_text = tokenizer.tokenize(example_text[1])
print(tokenized_text)
roberta_input = tokenizer(example_text, padding='max_length', max_length = 30, truncation=True, return_tensors="pt")
# Transform tokens to vocabulary indexes
indexed_tokens = tokenizer.convert_tokens_to_ids(tokenized_text)
#RoBERTa WORKS WITH THESE REPRESENTATIONS OBTAINED FOR THE TOKENS
print(roberta_input['input_ids'])
print(roberta_input['attention_mask'])
```

```
#To transform the sequence of token indices to words in
the texts
example_text                                            =
tokenizer.decode(roberta_input.input_ids[0])
print(example_text)
example_text                                            =
tokenizer.decode(roberta_input.input_ids[1])
print(example_text)
```

EXAMPLE 4: Obtaining contextualized word embeddings using the pre-trained RoBERTa model

```
import torch
from transformers import RobertaTokenizer, RobertaModel
tokenizer=RobertaTokenizer.from_pretrained('roberta-
base')
#The max_length parameter depends on the texts' length
in the dataset
example_text = ["Yo it shirt he gave New York was funny.",
"my husband is sick, homemade chicken soup loading.",
"Witch raises wind to break up enemy's lumber pound.",
"i got a new shirt at work at it is the WORST material
ever", "wave is so poor the girls can't even dress up for
Halloween", "The theory of paint indicates feelings of
isolation in society.", "Wish sometimes I had access to
the ever elusive cock carousel..", "off probation
tomorrow ima be a free woman again, FUCK THE SYSTEM"]

roberta_input          =          tokenizer(example_text,
padding='max_length', max_length = 50, truncation=True,
return_tensors="pt")
model = RobertaModel.from_pretrained('roberta-base')
#Setting the evaluation mode, this option does not make
gradient updating
model.eval()

# Send the data to GPU
```

```
# roberta_input = roberta_input.to('cuda')
# model.to('cuda')

# Send the data to mps, cuda or cpu
roberta_input = roberta_input.to(device)
model.to(device)
with torch.no_grad():
  outputs = model(**roberta_input)
  # Transformers models always return tuples.
  # Here, the first element corresponds to the vectors
in the output of the last RoBERTa layer.
  encoded_layers = outputs[0]
  print(encoded_layers.size())
  #Here we obtain the embedding of the CLS tokens for
each input text.
  #This representation serves as a contextual embedding
of the texts.
  cls_vector = encoded_layers[:,0,:]
  print(cls_vector.size())
  #Vector associated with the CLS token of the first text
in the entry.
  cls_vector = cls_vector.cpu().detach().numpy()[0]
  print(len(cls_vector))
```

BETO and RoBERTa variants for the Spanish language

BETO [5] is an initiative that enables the use of the pre-trained BERT model for the Spanish language. BETO has a comparable size to the BERT-based model. In this case, a model with 12 encoders and 16 self-attention heads was utilized, with a hidden layer size of 1024. The model has 110 million parameters. For training the model, texts were gathered from various sources, including Wikipedia and Spanish-language sources from the OPUS Project. These sources encompass United Nations and Government journals, TED Talks, subtitles, news articles, and more.

The overall corpus size is comparable to the corpus used in training the original BERT model.

RoBERTa Spanish [6] is a transformer-based masked language model for the Spanish language. It is based on the RoBERTa base model and has been pre-trained using the largest Spanish corpus known to date, with a total of 570GB of clean and deduplicated text processed for this work, compiled from the web crawlings performed by the National Library of Spain from 2009 to 2019.

EXAMPLE 5: Tokenization with BETO Tokenizer

```
import torch
from transformers import BertTokenizer
modelname='dccuchile/bert-base-spanish-wwm-uncased'
tokenizer= BertTokenizer.from_pretrained(modelname)
#The max_length parameter depends on the texts' length in the dataset

#Examples of the tokenization performed by BETO·
example_text = ['Cada vez más ganas de ponerme rubia tonta',"Dios cómo odio a los tíos que hacen manspreading","La frente muy alta, la lengua muy larga y la falda muy corta","Mis respetos pa la mujeres no pueden ser tan sorneras ajajajaj","No gusto de mujer sumisa, me gusta mi mujer toposa y toxica😌", "Las mujeres rápidas y los caballos lentos arruinarán tu vida.","hoy dejo d ser varón trans para elegir el camino de la misandria", "mucho feminismo pero te le tiras a vagos con novia jsjsja mi vida"]


tokenized_text = tokenizer.tokenize(example_text[0])
print(tokenized_text)
tokenized_text = tokenizer.tokenize(example_text[1])
```

```
print(tokenized_text)
beto_input                                                =
tokenizer(example_text,padding='max_length',max_length
= 20,
                     truncation=True,
return_tensors="pt")
# Transform tokens to vocabulary indexes
indexed_tokens                                            =
tokenizer.convert_tokens_to_ids(tokenized_text)
#BETO WORKS WITH THESE REPRESENTATIONS OBTAINED FOR THE
TOKENS
print(beto_input['input_ids'])
print(beto_input['token_type_ids'])
print(beto_input['attention_mask'])
#To transform the sequence of token indices to words in
the texts
example_text = tokenizer.decode(beto_input.input_ids[0])
print(example_text)
example_text = tokenizer.decode(beto_input.input_ids[1])
print(example_text)
```

EXAMPLE 6: Obtaining contextualized word embeddings using the pre-trained BETO model

```
import torch
from transformers import BertTokenizer, BertModel
modelname='dccuchile/bert-base-spanish-wwm-uncased'
tokenizer=BertTokenizer.from_pretrained(modelname)
example_text = ['Cada vez más ganas de ponerme rubia tonta',"Dios cómo odio a los tíos que hacen manspreading","La frente muy alta, la lengua muy larga y la falda muy corta","Mis respetos pa la mujeres no pueden ser tan sorneras ajajajaj","No gusto de mujer sumisa, me gusta mi mujer toposa y toxica😌", "Las mujeres rápidas y los caballos lentos arruinarán tu vida.","hoy dejo d ser varón trans para elegir el camino de la misandria", "mucho feminismo pero te le tiras a vagos con novia jsjsja mi vida"]
#The max_length parameter depends on the  texts' length in the dataset

beto_input = tokenizer(example_text, padding='max_length', max_length = 50, truncation=True, return_tensors="pt")
model = BertModel.from_pretrained(modelname)
#Setting the evaluation mode, this option does not make gradient updating
model.eval()
# Send the data to GPU
beto_input = beto_input.to('cuda')
model.to('cuda')
# Send the data to mps, cuda or cpu
beto_input = beto_input.to(device)
model.to(device)
with torch.no_grad():
  outputs = model(**beto_input)
  # Transformers models always return tuples.
```

```
  # Here, the first element corresponds to the vectors
in the output of the last BETO layer.
  encoded_layers = outputs[0]
  print(encoded_layers.size())
  #Here we obtain the embedding of the CLS tokens for
each input text.
  #This representation serves as a contextual embedding
of the texts.
  cls_vector = encoded_layers[:,0,:]
  print(cls_vector.size())
  #Vector associated with the CLS token of the first text
in the entry.
  cls_vector = cls_vector.cpu().detach().numpy()[0]
  print(len(cls_vector))
```

We can use RoBERTa Spanish Tokenizer and obtain contextualized word embeddings using the pre-trained RoBERTa (Spanish) model. Follow the examples 3 and 4 for RoBERTa English model, changing to the Spanish model `'PlanTL-GOB-ES/roberta-base-bne'`

ACTIVITY

Implement text representations for the corpora *EXIST2024_EN_examples_mini.csv (English language) and EXIST2024_ES_examples_mini.csv (Spanish language)*. Each text in the corpus should be represented using two distinct **contextual** embedding-based methods:

ENGLISH

- *BERT ('bert-base-uncased')*
- *RoBERTa ('roberta-base')*

SPANISH

- *BERT (' dccuchile/bert-base-spanish-wwm-uncased ')*
- *RoBERTa (' PlanTL-GOB-ES/roberta-base-bne')*

For the contextual embeddings, the representation of each text is obtained using the CLS (Classification) vectors in transformer-based models. Additionally, transformer models do not require any preprocessing step; they can process raw text directly. It is important to emphasize that the most similar tweet pairs must be determined **separately** for each class: **'SEXIST'** and **'NON-SEXIST'**. Specifically, you should find the most similar pair of tweets among those classified as 'SEXIST' and another most similar pair among those classified as 'NON-SEXIST'. To measure the similarity between tweet representations, you must use the cosine_similarity function provided by scikit-learn.

TIPS

- Processing the entire dataset simultaneously, as in the examples, will not be feasible due to its size. It will be necessary to split it into **batches** (taking for example batch size 16). Have a look to the **example 7 of the notebook**.
- To concatenate tensors along an existing dimension, use a torch.cat().
- To join two pytorch tensors adding a new axis use torch.stack().
- **AutoModel** is a generic class of the *transformers* library that can load a model without specifying its architecture. Instead of using specific classes like BertModel or RobertaModel, you can call AutoModel.from_pretrained("model-name"), and it will automatically detect the correct architecture.
- An **AutoTokenizer** class also exists.